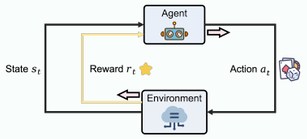
Reinforcement Learning Cheat Sheet (Basic Version)

October 26, 2026

Basic Knowledge

Basic Definition

Reinforcement Learning (RL): RL studies how an **agent** learns a **policy** for sequential decision-making (i.e., taking **actions**) by interacting with an **environment** and using **reward** feedback to maximize the expected **return**.



⋆ **State:** A state $s_t \in \mathcal{S}$ is a representation of the situation at time t that contains sufficient information for predicting the future interaction with environment.
State Space: The state space $\mathcal{S}$ is the set of all possible states.
⋆ **Action:** An action $a_t \in \mathcal{A}$ is a decision or operation made by the agent at time t, which is taken based on the current state s_t.
Action Space: The action space $\mathcal{A}$ is the set of all available actions in state s.

State Transition: The state transition is the process of moving from one state to another based on the current state and action taken by the agent. ($s_t \xrightarrow{a_t} s_{t+1}$)
Transition Probability: The transition probability $P(s' \mid s, a)$ specifies the probability of reaching the next state s' after taking action a in state s.

⋆ **Policy:** A policy π is a rule that specifies how the agent selects actions based on the current state. ($\pi(a \mid s) \geqslant 0, \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1$)
⋆ **Reward:** A reward $r_t \in \mathbb{R}$ is a scalar feedback signal provided by the environment after the agent takes action a_t in state s_t. (Immediate benefit of a_t)
⋆ **Trajectory:** A trajectory $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \ldots)$ is a sequence of states, actions, and rewards generated by the interaction between an agent and an environment. (example: $s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9$)
Episode: An episode $(S_0, A_0, R_1, \ldots, S_T)$ is a complete trajectory that starts from an initial state and ends when a terminal state (S_T) is reached.
Continuing Task: A continuing task is a task that does not have a natural terminal state and may continue indefinitely.

⋆ **Return:** The return G_t is the cumulative reward or total reward received from time t onward. ($G_t = \sum_{k=0}^{T-t-1} R_{t+k+1}$)
⋆ **Discounted Return:** The discounted return is the discounted sum of future rewards. ($G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, 0 \leq \gamma < 1$, where γ is the discount factor)

Bellman Equation and Bellman Optimality Equation

Bellman Equation 1: State Value and Action Value

⋆ **State Value:** The state value $v_\pi(s)$ is the expected return obtained by starting in state s and following policy π thereafter: $v_\pi(s) \triangleq \mathbb{E}_\pi[G_t \mid S_t = s]$.
State-wise Policy Dominance: Policy π_1 is at least as good as policy π_2 at state s if $v_{\pi_1}(s) \geqslant v_{\pi_2}(s)$. Policy π_1 dominates policy π_2 if $v_{\pi_1}(s) \geqslant v_{\pi_2}(s), \forall s \in \mathcal{S}$.

⋆ **Action Value:** The action value $q_\pi(s, a)$ is the expected return obtained by taking action a in state s and following policy π thereafter: $q_\pi(s, a) \triangleq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$.

Relation: ① Action value → State value: $v_\pi(s) = \sum_a \pi(a|s) q_\pi(s, a)$.
② State → Action value: $q_\pi(s, a) = \sum_{r,s'} p(r, s'|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_\pi(s')$.

Bellman Equation 2: Element-wise and Matrix-vector Form

⋆ **Core Idea:**

$$G_t = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \cdots) = \underbrace{R_{t+1}}_{\text{immediate reward}} + \gamma \cdot \underbrace{G_{t+1}}_{\text{future reward}}$$

$$\Rightarrow v_\pi(s) = \mathbb{E}[G_t|S_t = s] = \mathbb{E}[R_{t+1}|S_t = s] + \gamma \mathbb{E}[G_{t+1}|S_t = s].$$

⋆ **Bellman Equation (BE) (Element-wise Form):** (by Richard E. Bellman)

$$v_\pi(s) = \sum_a \pi(a|s) \left[\sum_{r,s'} p(r, s'|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s') \right], \quad \forall s \in \mathcal{S}.$$

Or:

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s'} p_\pi(s'|s) v_\pi(s'),$$

where $r_\pi(s) \triangleq \sum_a \pi(a|s) \sum_{r,s'} p(r, s'|s, a) r$ is the mean of immediate rewards; $p_\pi(s'|s) \triangleq \sum_a \pi(a|s) p(s'|s, a)$.

⋆ **Bellman Equation (BE) (Matrix-vector Form):**

$$\boldsymbol{v}_\pi = \boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_\pi,$$

$$\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{bmatrix} = \begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ \vdots \\ r_\pi(s_n) \end{bmatrix} + \gamma \begin{bmatrix} p_\pi(s_1|s_1) & \cdots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & \cdots & p_\pi(s_n|s_2) \\ \vdots & \ddots & \vdots \\ p_\pi(s_1|s_n) & \cdots & p_\pi(s_n|s_n) \end{bmatrix} \begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}$$

Solve the BE: ① Closed-form solution: $\boldsymbol{v}_\pi = (\boldsymbol{I} - \gamma \boldsymbol{P}_\pi)^{-1} \boldsymbol{r}_\pi$. (Inverse is hard)
② Iterative solution: $\boldsymbol{v}_{k+1} = \boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_k, \quad k = 0, 1, 2, \ldots. \quad (\boldsymbol{v}_k \xrightarrow{k \to \infty} \boldsymbol{v}_\pi)$

Bellman Optimality Equation

Better Policy: Policy π_1 is better than π_2 if $v_{\pi_1}(s) \geqslant v_{\pi_2}(s), \forall s \in \mathcal{S}$.
⋆ **Optimal Policy:** Policy π^* is a optimal policy if $v_{\pi^*}(s) \geqslant v_\pi(s), \forall s \in \mathcal{S}, \forall \pi$. The state values of π^* are optimal state values.

⋆ **Bellman Optimality Equation (BOE) (Element-wise Form):**

$$v^*(s) = \max_a \left[\bar{r}_\pi(s, a) + \gamma \sum_{s'} p_\pi(s'|s) v_\pi(s') \right] \triangleq \max_\pi \sum_{a \in \mathcal{A}} \pi(a|s) q(s, a), \quad \forall s \in \mathcal{S},$$

where $\bar{r}(s, a) \triangleq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$.
⋆ **Bellman Optimality Equation (BOE) (Matrix-vector Form):**

$$\boldsymbol{v}^* = \max_\pi (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}^*).$$

where $[\boldsymbol{r}_\pi]_s \triangleq \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r,s'} p(r, s'|s, a) r$, $[\boldsymbol{P}_\pi]_{s,s'} \triangleq \sum_{a \in \mathcal{A}} \pi(a|s) p(s'|s, a)$.

Existence & Uniqueness: BOE has a unique solution (fixed point), which can be approached iteratively by: $\boldsymbol{v}_{k+1} = T^* \boldsymbol{v}_k = \max_\pi (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_k)$.
Greedy Policy: $\max v = \max_\pi \sum_{a \in \mathcal{A}} \pi(a|s) q(s, a) = \max_{a \in \mathcal{A}(s)} q(s, a)$.

$$\pi(a \mid s) = \begin{cases} 1, & a = a^* \\ 0, & a \neq a^* \end{cases}, \quad a^* \in \arg\max_a q(s, a)$$

$\Rightarrow v_{k+1}(s) = \max_{a \in \mathcal{A}(s)} q(s, a)$.

Value Iteration & Policy Iteration

⋆ **Value Iteration:** $\boldsymbol{v}_{k+1} = \max_\pi (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_k)$. [Policy Update + Value Update]
① **Policy Update (PU):** Given $\boldsymbol{v}_k, \boldsymbol{r}_k, P_\pi \rightarrow$ update new policy π_{k+1}:

$$\pi_{k+1} = \arg\max_\pi (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_k), \quad (Greedy)\ \pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases},$$

where $a_k^*(s) = \arg\max_a q_k(s, a)$.

② **Value Update (VU):** Update state value $\boldsymbol{v}_{k+1}$ by π_{k+1}:

$$\boldsymbol{v}_{k+1} = \boldsymbol{r}_{\pi_{k+1}} + \gamma \boldsymbol{P}_{\pi_{k+1}} \boldsymbol{v}_k, \quad (Greedy)\ v_{k+1}(s) = \max_a q_k(s, a), s \in \mathcal{S}.$$

Note: $\boldsymbol{v}_k$ is a mid-estimation, but not a real state value under any policy.
Process: Estimation for $v_k(s) \rightarrow$ one-step look-ahead $q_k(s, a) = \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_k(s') \rightarrow$ greedy policy $a_k^*(s) = \arg\max_a q_k(s, a) \rightarrow$ value update $v_{k+1}(s) = \max_a q_k(s, a)$. *Value Iteration focuses on* ***state value****.*

⋆ **Policy Iteration:** Update a real policy till its optimality. [Policy Evaluation + Policy Improvement] *Policy Iteration focuses on* ***policy****.*
① **Policy Evaluation (PE):** Given policy $\pi_k \rightarrow$ solve the Bellman Equation $\boldsymbol{v}_{\pi_k} = \boldsymbol{r}_{\pi_k} + \gamma \boldsymbol{P}_{\pi_k} \boldsymbol{v}_{\pi_k}$ / $v_{\pi_k}(s) = \sum_a \pi_k(a|s) (\sum_{r,s'} p(r, s'|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s')) = \sum_a \pi_k(a|s) q_\pi(s, a) \rightarrow$ get state value $\boldsymbol{v}_{\pi_k}$

② **Policy Improvement (PI):** Given the real state value $\boldsymbol{v}_{\pi_k} \rightarrow$ update the policy (greedy) $\rightarrow$ new policy π_{k+1}:

$$\pi_{k+1} = \arg\max_\pi (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_{\pi_k}), \quad \pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases},$$

where $a_k^*(s) = \arg\max_a q_{\pi_k}(s, a)$.

Truncated Policy Iteration: Use limited iteration to solve the Bellman Equation $\boldsymbol{v}_{\pi_k} = \boldsymbol{r}_{\pi_k} + \gamma \boldsymbol{P}_{\pi_k} \boldsymbol{v}_{\pi_k}$.

Model-based Learning → Model-free Learning

Monte Carlo Learning

Idea: Model-based $v_\pi(s) = \sum_a \pi(a|s) q_\pi(s, a) = \sum_a \pi(a|s) (\sum_{r,s'} p(r, s'|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_\pi(s'))$, where $p(r, s'|s, a)$ and $p(s'|s, a)$ are given by the model $\rightarrow$ **Model-free** $q_{\pi_k}(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a)$.

⋆ **MC Basic:** Model-free Policy Iteration ① Policy Evaluation: $v_{\pi_k}(s) = \sum_a q_{\pi_k}(s, a), q_{\pi_k}(s, a) \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a)$, where $g^{(i)}(s, a) = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \gamma^2 r_{t+3}^{(i)} + \cdots, i \in [N]$ are samples ② Policy Improvement (Greedy): $\pi_{k+1}(s) \in \arg\max_\pi \sum_a \pi(a|s) q_{\pi_k}(s, a)$. *Note:* Each trajectory just as one sample for one g. Data efficient MC Basic: One trajectory from $s \rightarrow$ Multiple sub-trajectories from $s, s', \ldots$, i.e., use one trajectory to calculate multiple $q_{\pi_k}(\cdot)$.

MC ϵ-Greedy: MC without Exploring Starts: ① Idea: one Greedy PI trajectory may not cover all states $\rightarrow \epsilon$-Greedy PI to possibly explore all states
② ϵ-Greedy: $\pi(a \mid s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{for the greedy action} \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{for the other actions} \end{cases}$

Non-incremental Learning → Incremental Learning

Stochastic Approximation and Stochastic Gradient Descent

Idea: Calculation after data collected → Calculation during data is collecting
Problem Formulation: Solve $f(w) = 0$ with noisy output $\tilde{f}(w, \eta) = f(w) + \eta$

Robbins-Monro (RM) Algorithm: $w_{k+1} = w_k - a_k \tilde{f}(w_k, \eta_k), \quad k = 1, 2, 3, \ldots,$ where w_k is the k-th estimate of the solution.

Convergence Analysis: $w_k \xrightarrow{w.p.1} w^*$ if (1) $\forall w, 0 < c_1 \leqslant \nabla_w f(w) \leqslant c_2$ (2) $\sum_{k=1}^{\infty} a_k = \infty, \sum_{k=1}^{\infty} a_k^2 < \infty$ and (3) $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0, \mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty, \mathcal{H}_k = \{w_k, w_{k-1}, \ldots\}$.

(Full Batch) Gradient Descent (GD): $\min_w \ J(w) = \mathbb{E}[f(w, X)]$
$\rightarrow w_{k+1} = w_k - \alpha_k \nabla_w \mathbb{E}[f(w_k, X)] = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)]$.
Mini-batch Gradient Decent (MBGD): $w_{k+1} = w_k - \alpha_k \frac{1}{B} \sum_{i=1}^{B} \nabla_w f(w_k, x_i)$.
Stochastic Gradient Decent (SGD): $w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k)$.

Temporal-Difference Learning

Idea: *MC learning* uses N trajectories $(s_0^{(i)}, r_1^{(i)}, s_1^{(i)}, \ldots, s_t^{(i)}, r_{t+1}^{(i)}, s_{t+1}^{(i)}, \ldots), i \in [N]$ for one action value $q_{\pi_k}(s, a) \approx \frac{1}{N} \sum_{i=1}^{N} g^{(i)}(s, a), g^{(i)}(s, a) = \sum_{j=1} \gamma^{j-1} r_j^{(i)}$.
TD learning incrementally update like RM algorithm during data collection.

Basic TD: ① *Idea:* State value calculation by incremental update ② *Target:* $v_\pi(s_t) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t\right] \Rightarrow \tilde{f}(v_\pi) = v_\pi(s_t) - (r_{t+1} + \gamma v_\pi(s_{t+1}))$ ③ *Data:* $\{(s_t, r_{t+1}, s_{t+1})\}_t$ ④ *Algorithm:*

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t) \Bigg[\overbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD target } \bar{v}_t}}^{\text{TD error } \delta_t} \Bigg].$$

Sarsa: ① *Idea:* Action value calculation by incremental update + Policy Iteration ② *Target:* $q_\pi(s_t, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S', A') | s, a] \Rightarrow \tilde{h}(q_\pi) = q_\pi(s_t, a_t) - (r_{t+1} + \gamma q_\pi(s_{t+1}, a_{t+1}))$ ③ *Data:* $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$ ④ *Algorithm:*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\right]$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t), t = 0, 1, 2, \ldots.$$

Expected Sarsa: ① *Idea:* $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}) \rightarrow r_{t+1} + \gamma \mathbb{E}[q_t(S_{t+1}, A)]$ to decrease variance in Sarsa ② *Algorithm:*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)])\right]$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t).$$

n-step Sarsa: ① *Idea:* Use n-step look-ahead $G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n})$ ② *Algorithm:*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})]\right],$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t).$$

Q-Learning: ① *Idea:* TD Learning of optimal action values ② *Algorithm:*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)]\right]$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t).$$

Behavior Policy: for executing actions and generating experience samples **Target Policy:** for being evaluated and improved for final putting into practice.
On-policy: behavior policy = target policy
Off-policy: behavior policy ≠ target policy

Tabular Learning → Function Learning

Value Function Approximation

Tabular Learning: Use tables to represent discrete state and action values.

s_1	$\cdots$	s_9
$v_\pi(s_1)$	$\cdots$	$v_\pi(s_9)$

State / Action	a_1	$\cdots$	a_5
s_1	$q_\pi(s_1, a_1)$	$\cdots$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\cdots$	$\vdots$

Function Approximation: Use continuous functions that parameteried by a small number parameters to represent the state and the action-value function:

$$\hat{v}(s, w) = \phi^T(s) w, \quad \hat{q}(s, a, w) = \phi^T(s, a) w.$$

Key Advantage: ① Smaller storage ② Generalization ability.

Function Approximation based TD Learning: ① Objective Function: $J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$ ② Minimization: $w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$, $\nabla_w J(w) = -2\mathbb{E}[(v_\pi(S) - \hat{v}(S, w)) \nabla_w \hat{v}(S, w)]$:

$$\begin{aligned} &GD: \quad w_{k+1} = w_k + 2\alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w_k)) \nabla_w \hat{v}(S, w_k)], \\ &SGD: \quad w_{t+1} = w_t + 2\alpha_t \left(v_\pi(s_t) - \hat{v}(s_t, w_t)\right) \nabla_w \hat{v}(s_t, w_t). \end{aligned}$$

TD-Linear: $v_\pi(s_t) \leftarrow r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t)$, $v_\pi(s_t) \leftarrow \phi^T(s) w$:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t\right] \phi(s_t).$$

Function Approximation based Sarsa:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)\right] \nabla_w \hat{q}(s_t, a_t, w_t).$$

Deep Q-learning / Deep Q-network (DQN): ① Objective Function: $J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right]$. ③ Idea: (1) Optimize main network $\hat{q}(s, a, w)$ to approximate q_π (2) Use target network $\hat{q}(s, a, w_T)$ as target, and update w every T time:

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right].$$

Experience Replay: Data $(S_0, A_0, R_1, S_1), (S_1, A_1, R_2, S_2), (S_2, A_2, R_3, S_3), \ldots$ from trajectories are temporally sequential and highly correlated → Store in the replay buffer $\mathcal{B} \triangleq \{(s, a, r, s')\}$ → Randomly sample for training.

Policy Gradient 1: Parameterized Policy

Tarbular Policy: Use table to represent policy π:

State / Action	a_1	$\cdots$	a_5
s_1	$\pi(a_1 \mid s_1)$	$\cdots$	$\pi(a_5 \mid s_1)$
$\vdots$	$\vdots$	$\ddots$	$\vdots$
s_9	$\pi(a_1 \mid s_9)$	$\cdots$	$\pi(a_5 \mid s_9)$

Parameterized Policy: Represent policy by a differentiable function $\pi_\theta(a \mid s)$.
Key Advantages: ① Compact representation (Storage ↓) ② Generalization.

Policy Gradient

Metrics to Define Optimal Policies: ① Average (state) value:

$$\bar{v}_\pi = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) = \boldsymbol{d}^T \boldsymbol{v}_\pi, \boldsymbol{v}_\pi = [\ldots, v_\pi(s), \ldots]^T \in \mathbb{R}^{|\mathcal{S}|},$$

where $\boldsymbol{d} = [\ldots, d(s), \ldots]^T \in \mathbb{R}^{|\mathcal{S}|}$ is the weight and $d(s) \geqslant 0$, $\forall s, \sum_{s \in \mathcal{S}} d(s) = 1$.
② Average (one-step) reward:

$$\bar{r}_\pi \triangleq \sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}[r_\pi(S)], \quad r_\pi(s) \triangleq \sum_{a \in \mathcal{A}} \pi(a|s) r(s, a),$$

where $S \sim d_\pi$ is stationary distribution, $r_\pi(s)$ is immediate reward and $r(s, a) = \mathbb{E}[R|s, a] = \sum_{r, s'} r p(r, s'|s, a)$.

Objective Function: Maximize the metrics:

$$\text{Average Value:} J(\theta) = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right],$$

$$\text{Average Reward:} J(\theta) = \sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s) = \lim_{n \to \infty} \frac{1}{n} \mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} | S_t = s_0\right].$$

Policy Gradient Theorem: ($\nabla_\theta \ln \pi(a|s, \theta) = \frac{\nabla_\theta \pi(a|s, \theta)}{\pi(a|s, \theta)}$)

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s \in \mathcal{S}} \rho_{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s, \theta) q_\pi(s, a), \\ &= \sum_s \rho_{\pi_\theta}(s) \sum_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) q_{\pi_\theta}(s, a), \end{aligned}$$

where $\rho_{\pi_\theta}(s) = \mathbb{E}_{d_0, \pi_\theta}[\sum_{t=0}^{T-1} \gamma^t \mathbf{1}\{S_t = s\}]$ is the discounted state occupancy.

Gradient Ascent: $\theta_{k+1} = \theta_k + \alpha_k \nabla_\theta J(\theta_k)$.
REINFORCE (Monte Carlo Policy Gradient): ① Generate an episode using π_{θ_k}; ② Estimate action values using returns; ③ Update the policy:

$$G_t = \sum_{j=t}^{T-1} \gamma^{j-t} R_{j+1}, \quad \theta_{k+1} = \theta_k + \alpha_k \sum_{t=0}^{T-1} \gamma^t G_t \nabla_\theta \log \pi_{\theta_k}(A_t \mid S_t).$$